guildtrek

PYTHON FOR AI AND DATA ENGINEERING · DAY 1

Day 1, Python in context and a real environment

A demo runs once on a laptop. **An enterprise app has to ship, survive a second year, and be handed to a team.**

By the end of today you can explain what Python really is and set up a project a team can trust from the first commit.

PART 1 · PYTHON IN CONTEXT

What Python is, and how the runtime really behaves

Build the model of the machine before you write a line of application code.

Where Python fits, and the honest why not

Where it wins

- AI, machine learning, and data science
- Automation, scripting, and glue
- Web and API backends, input and output bound
- Data engineering pipelines, extract, transform, load
- Prototyping and research

Python wins where engineering time is the scarce resource.

Where it is the wrong tool

- Hard real time and microsecond latency
- CPU bound work that must saturate all cores in one process
- Systems programming: drivers, firmware
- Mobile and browser front ends
- Tiny, memory constrained devices

Not a broken tool. A mismatch.

ENTERPRISE LENS Knowing the weak spots is risk management. Plan for a native module or a separate service early, before load finds the limit in production.

The move that makes the weakness disappear

Python is the calm, readable surface. The fast work happens in native libraries underneath. This is the two-language pattern.

Your idea → Readable Python logic → calls → C, C++, CUDA inner loops

Level	Languages	What lives here
Hardware and CPU	machine code	The processor understands only binary instructions.
Operating system and drivers	C, C++, Rust	Compiled to machine code, manual memory, ultra low latency.
Virtual machine and JIT	Java, C#, Go	Bytecode on a managed runtime, high concurrency backends.
Interpreted and scripting	Python, Ruby, JavaScript	Highest abstraction, memory managed, built to command the layers below.

> When you multiply two large arrays with NumPy, almost no time is spent in Python. "Python is slow" always deserves the follow up: slow for what, and can the hot part go to a library that is not slow at all?

What Python really is: compiled, then interpreted

Not one or the other. **Python is compiled to bytecode, then interpreted by a virtual machine.**

Your source → compile step → bytecode → virtual machine, a C loop → it runs

What the compile step buys

Quick, automatic, invisible. This is the fast startup and the interactive feel.

What the interpreter costs

The virtual machine runs bytecode one instruction at a time, not your processor directly. That is the cost on a tight numeric loop.

THE FOLDER YOU WILL SEE The first import caches bytecode in `__pycache__` as a `.pyc` file. Safe to delete, belongs in your ignore file, never in version control.

CPython, and why you pin the version

The Python language is a spec. **CPython** is the program that runs it, written in C. Its C interface is exactly why the AI and data stack chose it.

Implementation	In	Why it exists
CPython	C	The reference. The default.
PyPy	Python	A JIT, faster pure Python.
Jython, IronPython	Java, C#	Python on the JVM and .NET.
MicroPython	C	Tiny Python for microcontrollers.

Read a version like 3.11.5

- **3.11** is the language level, effectively a slightly different language each minor step.
- **5** is a patch, bug and security fixes only, always safe to take.

This program means CPython 3.11 or newer. A version manager picks the interpreter, a project manager locks the packages.

ENTERPRISE LENS An unpinned version is a time bomb: rebuild next year on 3.13, a dependency behaves differently, and a bug shows up in production nobody can reproduce.

The GIL: threads for waiting, processes for computing

The single most important runtime fact about CPython: the **Global Interpreter Lock** lets only one thread run Python bytecode at a time.

I/O bound work: threads help

Spends its time *waiting* for a network reply, a query, a file. A waiting thread releases the GIL, so many requests run in flight at once.

CPU bound work: threads do not

Spends its time *computing*. Only one thread holds the GIL, so more threads just take turns on one lock, and can even slow things down.

To use all cores for heavy computation, use **processes, not threads**. Each process has its own interpreter and its own GIL. Native libraries like NumPy also release the GIL while their C loops run.

THE EXPENSIVE MISTAKE A team adds a thread pool to a slow, CPU bound data job and sees no speedup. Knowing the split up front picks the right tool: a process pool for the heavy transform, threads for the many small waits.

PART 2 · THE TOOLCHAIN

Dev now, prod in mind

Not one Python you install and you are done. A small stack of tools, each doing one job, set up toward what production actually looks like.

The Python ecosystem: a stack of tools, by job

Get the map before you install anything. From the surface you touch down to what runs your code.

Layer (what it does)	Tool you will use	Its job
Where you work	VS Code (or PyCharm)	write, navigate, and debug code
Code quality	ruff, mypy, pre-commit	keep code correct, typed, and consistent
Dependencies and the project	Poetry	declare and lock what the project depends on
Native and GPU libraries	conda or mamba	install system C, CUDA, and BLAS that pip cannot
Environment isolation	venv	keep each project's packages separate
Python versions	pyenv	install and pin the interpreter version
The interpreter	CPython	actually runs your bytecode
Package and ship	Docker	one image that runs the same everywhere

EACH TOOL EARNS ITS PLACE Nothing here is busywork. Each one is present because a real thing breaks without it. We set them up top to bottom.

Development and production are two different worlds

A script on your laptop and a service in production are not the same thing on a bigger machine. Here is the honest picture of production.

Immutable and ephemeral

Runs from a fixed image. You cannot SSH in to fix a file. Misbehave and the orchestrator destroys it and starts fresh. State lives in a database, never on the container.

Orchestration and autoscaling

The platform runs many identical copies and starts or stops them with traffic. Your app must be fine with any one of them vanishing.

Structured logging

Nobody watches your terminal, there is no terminal. Logs stream as JSON records to a central system. A bare `print` is invisible.

No dev tools inside, and CI and CD gates

The image carries only what it runs. The linter, type checker, and tests ran earlier in the pipeline. Code ships only after the gates pass.

ENTERPRISE LENS Each constraint is a design instruction for today: read config from the environment, keep no state in a local variable across requests, reach for a logger, treat the quality gate as the gate to ship.

The IDE: where you work

The top of the stack, worth setting up once, well.

VS Code, the common enterprise choice

Free, light, extensible, with Remote SSH and Dev Containers so you edit locally while code runs on a remote GPU box. PyCharm is a fine, heavier alternative.

Extensions do the real work

- **Python** for language support and running code
- **Ruff** for lint and format on save
- **mypy** for type feedback as you type
- **Jupyter** for notebooks

COMMIT THE TEAM SETTINGS A `.vscode/settings.json` in the repo makes everyone's editor behave the same: format on save with Ruff, point at the project's own interpreter. New teammates get the house behaviour for free.

Code quality: the gate that blocks a bad commit

Python catches many mistakes at runtime, not at a compile step. The quality gate buys those guarantees back. Three tools working together.

ruff

Linter and formatter in one, written in Rust, so fast it runs on every save. Replaces flake8, black, and isort.

mypy, strict mode

Static type checker. Reads your annotations and checks them **without running the code**. Strict mode insists every function is typed.

pre-commit

The enforcer. Runs ruff and mypy **before** a commit completes. A problem, and the commit is blocked.



A linter you have to remember to run is a linter you will forget. Wired into the commit, broken code physically cannot enter the repository.

Dependencies: Poetry, not pip

The weakness of `pip freeze > requirements.txt` is **transitive dependencies**: it mixes the packages you chose with the ones they dragged in, and often does not lock them, so a build changes under you.

pyproject.toml declares

The handful of packages you truly depend on, with version ranges you allow. You edit it, or let `poetry add` edit it.

poetry.lock locks

The exact resolved graph, every package direct and transitive, pinned with a checksum. Poetry generates it, you **commit it to Git**.

```
poetry add python-dotenv          # add a dependency, update pyproject.toml and the lock
poetry add --group dev ruff mypy  # add tools to the dev group, not the shipped deps
poetry install                    # create the venv and install the exact locked graph
poetry run pytest -q              # run a command inside the project's environment
```

ONE LINE ON THE ALTERNATIVE uv, from Astral, does the same declare and lock job much faster and is gaining ground. We teach Poetry because it is mature and widely taught, and the concepts carry straight over.

conda versus pip: a different problem

Not a rival to Poetry. It solves a different problem, and the clean rule is to know which world you are in.

pip and Poetry install Python packages

They pull Python code as wheels from PyPI. When a package needs a bit of compiled C, it usually ships a prebuilt wheel and you never notice. This covers almost everything in this program.

conda installs system level things pip cannot

CUDA drivers for a GPU, BLAS for fast linear algebra, GDAL for geospatial. Those are system C and C++ libraries, not Python. conda installs Python, the packages, and those native libraries together.

ALSO MEET

- **Miniconda, the small install**
- **Mamba, a faster solver in C++**
- **Micromamba, tiny and standalone**

THE RULE Poetry by default. Reach for conda only when the wheel is not enough and you need an exact CUDA or BLAS that pip cannot provide.

Isolation and the interpreter: venv and pyenv

Two layers under the package manager: the isolated environment your packages live in, and the interpreter version that runs them.

venv, managed by Poetry

An isolated, per project set of packages, separate from the system Python and every other project. Poetry creates and manages one per project. You never make it by hand.

pyenv, picks the version

Installs and switches Python versions, so you do not depend on whatever the operating system ships. Pick a supported version on purpose and pin it.

THE RULE THAT NEVER CHANGES Never install project packages into the system Python. Isolation plus a pinned version is what makes "works on my machine" actually mean something.

Containers, for dev and prod parity

A venv isolates packages on one machine. A **container** goes further: it packages the interpreter, the dependencies, and your code into one image that runs the same anywhere.

Image

A built, read only snapshot in layers: a base OS, a specific Python, your dependencies, and your code.

Container

A running instance of an image, isolated from the host. The immutable, ephemeral unit from the production picture.

For local development you often need more than the app: a database, a cache, a queue. **Docker Compose** declares them in one file and starts them together, so your laptop runs the same Postgres production will.

```
docker compose up -d
```

ENTERPRISE LENS The same Postgres version on your laptop as in prod shrinks the whole class of "it behaved differently in prod" bugs. Worth building toward from Day 1, before today's code even needs the database.

PART 3 · THE BUILD

The reviews-insight skeleton

Not a demo. A skeleton for an enterprise app, shaped correctly, with a lockfile, a quality gate, and a database service ready.

The project shape

A demo is one `app.py`. A real project has a shape, because a shape is what lets it grow and be handed over.

```
reviews-insight/
  pyproject.toml            # metadata, deps, tool config
  poetry.lock               # the exact graph, committed
  .gitignore                # what never gets committed
  .env.example              # config template, safe to commit
  .pre-commit-config.yaml   # the commit gate
  docker-compose.yml        # a Postgres database
  src/
    reviews_insight/        # the importable package
      __init__.py
      config.py             # settings from the environment
      data.py               # data loading
      cli.py                # the command line entry point
  tests/
    test_summary.py         # tests, kept separate
  data/
    reviews_sample.csv      # sample data
```

A src/ layout

Stops accidental imports before install, so tests exercise the package the way a user would. What `poetry new --src` gives you.

Tests are separate

`tests/` sits beside the source. A first class part of the project, not an afterthought.

Config is its own module

`config.py` is the one place settings are read. Nothing else reads the environment directly.

Scaffold, install, and lock

Poetry builds the whole layout, then resolves and pins the exact graph.

```
poetry new --src reviews-insight   # the --src flag asks for the src/ layout
cd reviews-insight
poetry install                     # write poetry.lock, make the venv, install everything
```

pyproject.toml declares

A short, honest runtime list: `python = "^3.11"` and `python-dotenv`. A separate dev group holds ruff, mypy, pytest, and pre-commit, so they never ship in the image.

A real console command

`[tool.poetry.scripts]` maps `reviews` to `reviews_insight.cli:main`. After install, `reviews` is a real command, not `python some/path/script.py`.

ENTERPRISE LENS `poetry install` against a committed lock is the same operation your build server runs. The laptop and the pipeline build from the identical pinned graph. Reproducibility you can point at.

Config and secrets from the environment

Never hard code settings, and never commit secrets. Config that changes between environments is read in one place, so the same code runs in dev, test, and prod with only the environment changing.

```python
@dataclass(frozen=True)
class Config:
    app_name: str
    data_path: str
    environment: str

    @classmethod
    def from_env(cls) -> Config:
        return cls(
            app_name=os.environ.get(...),
            data_path=os.environ.get(...),
            environment=os.environ.get(...),
        )
```

A frozen settings object

- A local `.env`, never committed, supplies values in dev
- Real environment variables supply them in prod
- A committed `.env.example` documents which variables exist, with no real secret

ENTERPRISE LENS A secret committed to Git is a secret leaked, permanently. Reading config from the environment is also what lets one image run in every environment, the backbone of a reliable pipeline.

The quality gate in action

Run the code through the gate, each tool through Poetry, then make it automatic with a commit hook.

Test

```
poetry run pytest -q
```

1 passed

Lint

```
poetry run ruff check .
```

All checks passed!

Type check

```
poetry run mypy src
```

Success: no issues found

Then wire ruff and mypy into Git as hooks, pinned to exact versions, and install once:

```
poetry run pre-commit install
```

WATCH IT WORK Introduce a type error, stage it, try to commit. The mypy hook finds the mismatch and the commit is blocked. Fix the type, commit again, every hook passes. The check is not a thing you remember, it is a gate broken code cannot pass.

What we covered

Python in context

- Where it fits, and the honest why not
- The two-language pattern: readable surface, native core
- Compiled to bytecode, then interpreted; CPython; pin the version
- The GIL: threads for waiting, processes for computing

The toolchain and the build

- The stack of tools by job, dev now and prod in mind
- Poetry with a committed lock, the ruff and mypy gate
- venv and pyenv, containers for parity
- The reviews-insight skeleton, run, tested, and guarded

THE THREAD OF THE DAY A real environment, a locked dependency graph, an enforced quality gate, and container parity are what separate a system that ships from a demo that ran once.

guildtrek

DAY 1 · COMPLETE

Set it up right on the first commit.

The gap between "it ran on my laptop" and "it ships" is exactly the set of habits you practiced today, on the very first project.

Next: Day 2, putting the environment to work on data itself.